Lab 4

Pipelining and Datapath

Task	Objective Description
Task 1: Behavioural Simulation	To examine the uP16 pipelined processor structure, modify instruction memory contents, synthesize the design, and verify correct operation using behavioural simulation. (I have only explained this in video)
Task 2: Post-Synthesis Timing Simulation	To perform timing simulation and compare it with behavioural simulation to observe propagation delay effects.
Task 3: FPGA Implementation	To implement the uP16 processor on the Basys 3 board and verify correct program execution via 7-segment display and LEDs.

16 bit microprocessor- no data forwarding

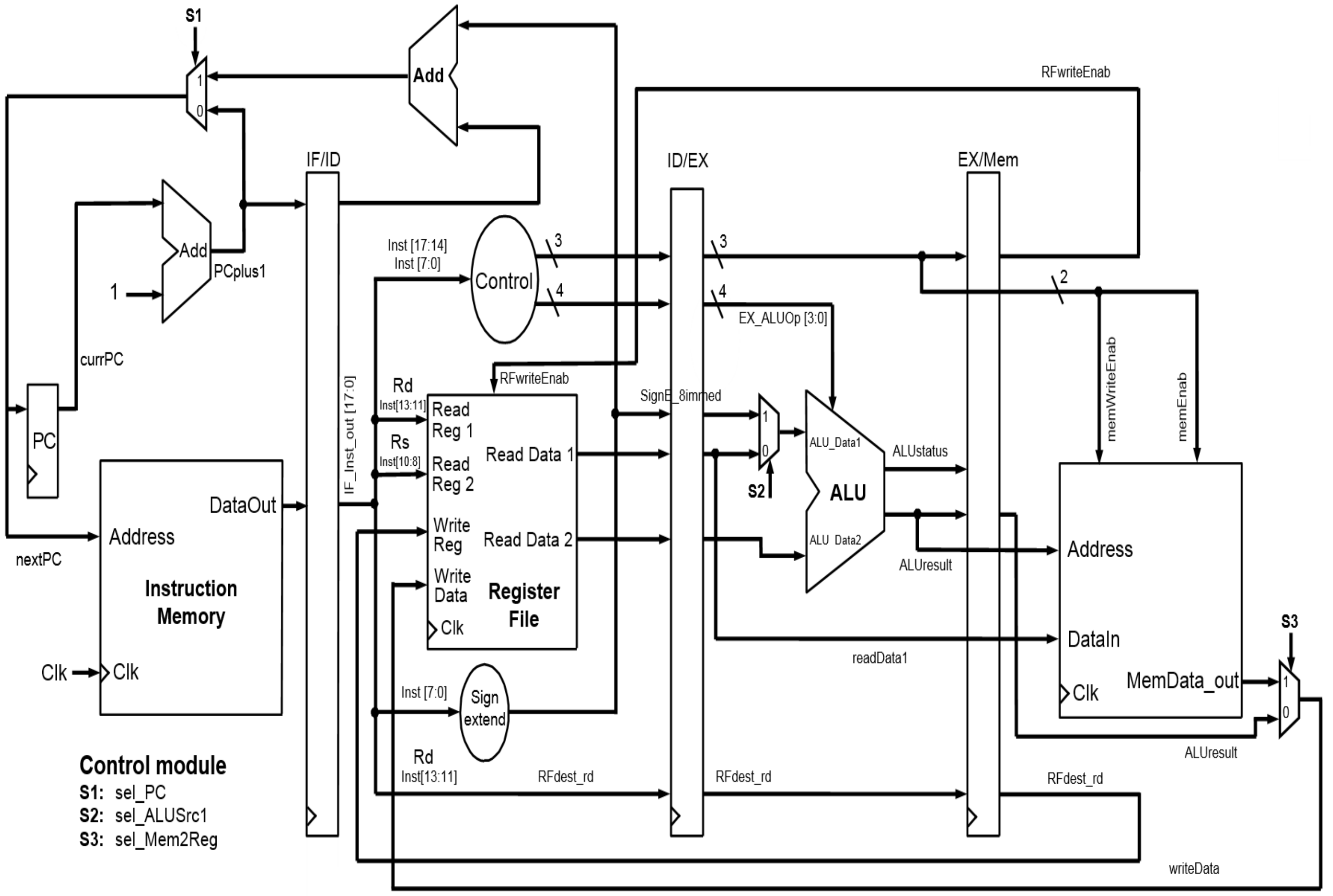


Table 1: The 18 bit instruction formats for the uP16 processor

Field	4 bits	3 bits	3 bits	8 bits
R-format	OpCode	Rd	Rs	Function/Immed

Table 2: The instruction set for the uP16 processor

Name	op	Funct/ Immed	Instruction	Effect(s)
nop	0x00	0x00	**nop**	Do nothing (instruction word = 0)
add	0x00	0x01	**add** *rd, rs*	RF[rd] = RF[rd] + RF[rs];
sub	0x00	0x02	**sub** *rd, rs*	RF[rd] = RF[rd] - RF[rs];
and	0x00	0x03	**and** *rd, rs*	RF[rd] = RF[rd] **and** RF[rs];
or	0x00	0x04	**or** *rd, rs*	RF[rd] = RF[rd] **or** RF[rs];
nand	0x00	0x05	**nand** *rd, rs*	RF[rd] = RF[rd] **nand** RF[rs];
nor	0x00	0x06	**nor** *rd, rs*	RF[rd] = RF[rd] **nor** RF[rs];
xor	0x00	0x07	**xor** *rd, rs*	RF[rd] = RF[rd] **xor** RF[rs];
not	0x00	0x08	**not** *rd, rs*	RF[rd] = **not** RF[rs];
mov	0x00	0x0A	**mov** *rd, rs*	RF[rd] = RF[rs];
lw	0x01	--	**lw** *rd, rs, immed*	RF[rd] <- memory(RF[rs]+immed);
sw	0x02	--	**sw** *rd, rs, immed*	RF[rd] -> memory(RF[rs]+immed);
lli	0x03	immed	**lli** *rd, rs, immed*	RF[rd] = sign_ext(immed); (*rs* is unused)
lui	0x04	immed	**lui** *rd, rs, immed*	RF[rd] = {immed, RF[rs][7:0]}
addi	0x05	immed	**addi** *rd, rs, immed*	RF[rd] = RF[rs] + sign_ext(immed);
	0x6-8			**Reserved**
beq	0x08	immed	**beq** *rd, rs, immed*	if (RF[rd] = = RF[rs]) pc += sign_ext(immed) + 1;
bne	0x09	immed	**bne** *rd, rs, immed*	if (RF[rd] != RF[rs]) pc += sign_ext(immed) + 1;
blt	0x0A	immed	**bltz** *rd, rs, immed*	if (RF[rd] < RF[rs]) pc += sign_ext(immed) + 1;
bgt	0x0B	immed	**bgtz** *rd, rs, immed*	if (RF[rd] > RF[rs]) pc += sign_ext(immed) + 1;
	0xC-F			**Reserved**

Table 1: The 18 bit instruction formats for the uP16 processor

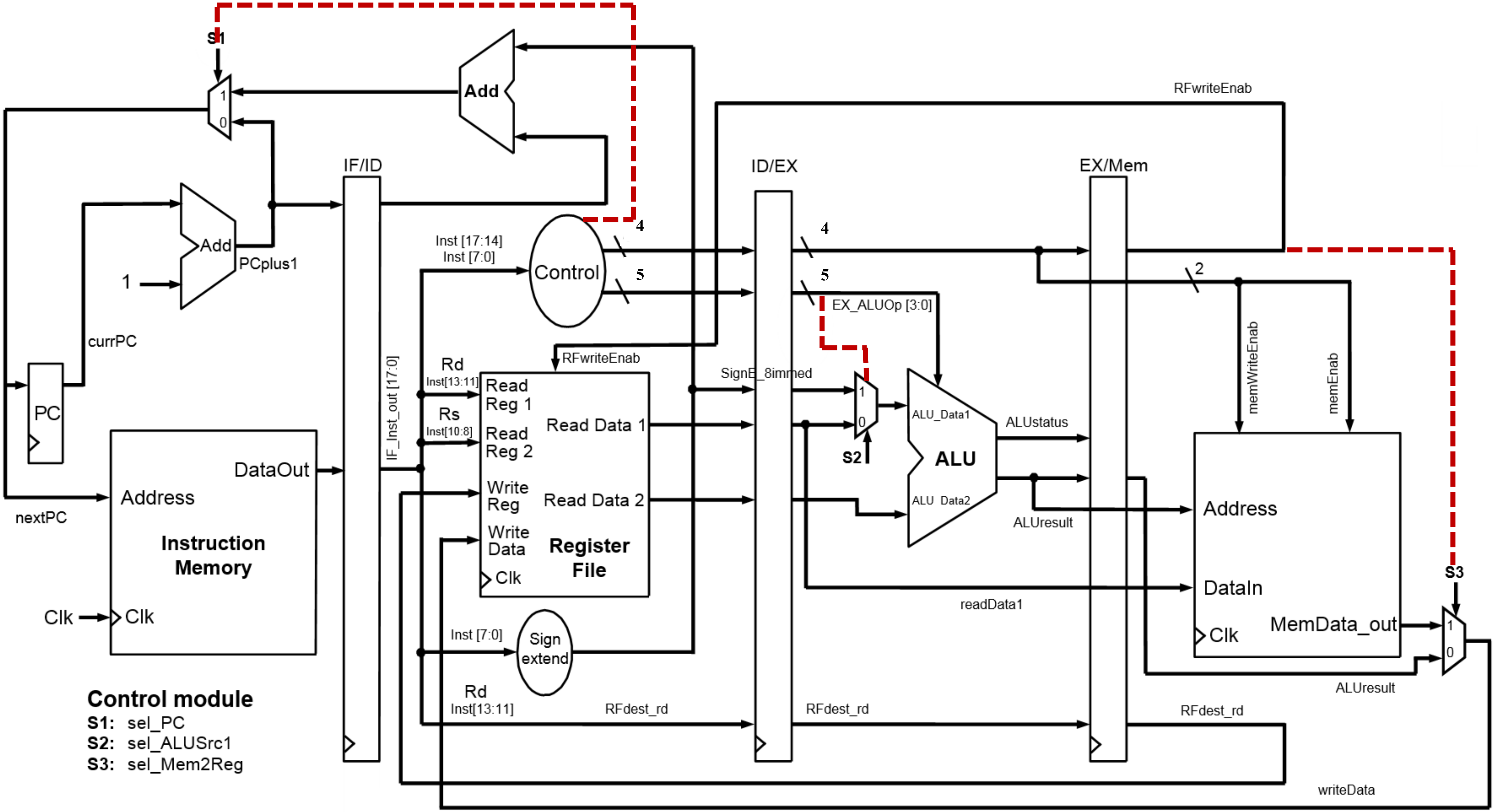
Field	4 bits	3 bits	3 bits	8 bits
R-format	OpCode	Rd	Rs	Function/Immed

Table 2: The instruction set for the uP16 processor

Name	op	Funct/ Immed	Instruction	Effect(s)
nop	0x00	0x00	**nop**	Do nothing (instruction word = 0)
add	0x00	0x01	**add** *rd*, *rs*	RF[rd] = RF[rd] + RF[rs];
sub	0x00	0x02	**sub** *rd*, *rs*	RF[rd] = RF[rd] - RF[rs];
and	0x00	0x03	**and** *rd*, *rs*	RF[rd] = RF[rd] **and** RF[rs];
or	0x00	0x04	**or** *rd*, *rs*	RF[rd] = RF[rd] **or** RF[rs];
nand	0x00	0x05	**nand** *rd*, *rs*	RF[rd] = RF[rd] **nand** RF[rs];
nor	0x00	0x06	**nor** *rd*, *rs*	RF[rd] = RF[rd] **nor** RF[rs];
xor	0x00	0x07	**xor** *rd*, *rs*	RF[rd] = RF[rd] **xor** RF[rs];
not	0x00	0x08	**not** *rd*, *rs*	RF[rd] = **not** RF[rs];
mov	0x00	0x0A	**mov** *rd*, *rs*	RF[rd] = RF[rs];
lw	0x01	--	**lw** *rd*, *rs*, *immed*	RF[rd] <- memory(RF[rs]+immed);
sw	0x02	--	**sw** *rd*, *rs*, *immed*	RF[rd] -> memory(RF[rs]+immed);
lli	0x03	immed	**lli** *rd*, *rs*, *immed*	RF[rd] = sign_ext(immed); (*rs* is unused)
lui	0x04	immed	**lui** *rd*, *rs*, *immed*	RF[rd] = {immed, RF[rs][7:0]}
addi	0x05	immed	**addi** *rd*, *rs*, *immed*	RF[rd] = RF[rs] + sign_ext(immed);
	0x6-8			**Reserved**
beq	0x08	immed	**beq** *rd*, *rs*, *immed*	if (RF[rd] = = RF[rs]) pc += sign_ext(immed) + 1;
bne	0x09	immed	**bne** *rd*, *rs*, *immed*	if (RF[rd] != RF[rs]) pc += sign_ext(immed) + 1;
blt	0x0A	immed	**bltz** *rd*, *rs*, *immed*	if (RF[rd] < RF[rs]) pc += sign_ext(immed) + 1;
bgt	0x0B	immed	**bgtz** *rd*, *rs*, *immed*	if (RF[rd] > RF[rs]) pc += sign_ext(immed) + 1;
	0xC-F			**Reserved**

For add/sub/and/or/nand/nor/xor/not and mov- opcode is 0x00 (shown in red) , hence ALUop from control logic uses the function field (in orange)

Inst	ALUop	sel_ALU src1	mem2 Reg	mem Enab	memWrite Enab	RFwrite Enab	PC_select
LW	0001	1	1	1	0	1	0
SW	0001	1	0	1	1	0	0
LLI	1011	1	0	0	0	1	0
LUI	1100	1	0	0	0	1	0
ADDI	0001	1	0	0	0	1	0
BEQ	X	X	0	0	0	0	1(readData1 == readData2)
BNE	X	X	0	0	0	0	1(readData1 != readData2)
BLT	X	X	0	0	0	0	1(readData1 < readData2)
BGT	X	X	0	0	0	0	1(readData1 > readData2)
default	0000	X	0	0	0	0	0


S1
Add
IF/ID
ID/EX
EX/Mem
RFwriteEnab
Add
PCplus1
1
currPC
PC
nextPC
Address
DataOut
Instruction
Memory
Clk
Clk
Inst [17:14]
Inst [7:0]
Control
4
5
4
5
EX_ALUOp [3:0]
2
IF_Inst_out [17:0]
Rd
Inst[13:11]
Rs
Inst[10:8]
RFwriteEnab
Read
Reg 1
Read
Reg 2
Write
Reg
Write
Data
Read Data 1
Read Data 2
Register
File
Clk
SignE_8immed
1
0
S2
ALU_Data1
ALU
ALU Data2
ALUstatus
ALUresult
readData1
memWriteEnab
memEnab
Address
DataIn
Clk
MemData_out
S3
1
0
ALUresult
Inst [7:0]
Sign
extend
Rd
Inst[13:11]
RFdest_rd
RFdest_rd
RFdest_rd
writeData
Control module
S1: sel_PC
S2: sel_ALUSrc1
S3: sel_Mem2Reg

Hierarchy after including all files for 16 bit processor

- **uP16_top** (uP16_top.v) (7)
  - T1 : IF_stage (IF_stage.v) (1)
  - T2 : IF_ID_PR (IF_ID_PR.v)
  - T3 : ID_stage (ID_stage.v) (2)
  - T4 : ID_EX_PR (ID_EX_PR.v)
  - T5 : EX_stage (EX_stage.v) (1)
  - T6 : EX_Mem_PR (EX_Mem_PR.v)
  - T7 : Mem_stage (Mem_stage.v) (1)

Sources

0

- Design Sources (1)
  - **uP16_top** (uP16_top.v) (7)
    - T1 : IF_stage (IF_stage.v) (1)
      - IF_1 : I_mem (Instruct_mem.v)
    - T2 : IF_ID_PR (IF_ID_PR.v)
    - T3 : ID_stage (ID_stage.v) (2)
      - EX_1 : Reg_File (ID_stage.v)
      - EX_3 : Control (ID_stage.v)
    - T4 : ID_EX_PR (ID_EX_PR.v)
    - T5 : EX_stage (EX_stage.v) (1)
      - EX_1 : ALU (EX_stage.v)
    - T6 : EX_Mem_PR (EX_Mem_PR.v)
    - T7 : Mem_stage (Mem_stage.v) (1)
      - DM_1 : D_mem (Data_mem.v)

Open instruct_mem.v and see BRAM MACRO for .INIT and .INITP.

```
// Xilinx HDL Libraries Guide, version 2012.2
/////////////////////////////////////////////////////////////////////
// READ_WIDTH   | BRAM_SIZE | READ Depth   | ADDR Width    |          //
// WRITE_WIDTH  |           | WRITE Depth  |               | WE Width //
// ============|===========|=============|============|================//
// 10-18        | "18Kb"    | 1024         | 10-bit        | 2-bit    //
/////////////////////////////////////////////////////////////////////
BRAM_SINGLE_MACRO #(
.BRAM_SIZE("18Kb"), // Target BRAM, "18Kb" or "36Kb"
.DEVICE("7SERIES"), // Target Device: "7SERIES"
.DO_REG(0), // Optional output register (0 or 1)
//.INIT(36'h000000000), // Initial values on output port
.INIT_FILE ("NONE"),
.WRITE_WIDTH(18), // Valid values are 1-72 (37-72 only valid when BRAM_SIZE="36Kb")
.READ_WIDTH(18), // Valid values are 1-72 (37-72 only valid when BRAM_SIZE="36Kb")
//.SRVAL(36'h000000000), // Set/Reset value forr port output
.WRITE_MODE("WRITE_FIRST"), // "WRITE_FIRST", "READ_FIRST", or "NO_CHANGE"
.INIT_00(256'h0000000000000000000000000000000000000000000000000000000000000000),
.INIT_01(256'h0000000000000000000000000000000000000000000000000000000000000000),
.INIT_02(256'h0000000000000000000000000000000000000000000000000000000000000000),
.INIT_03(256'h0000000000000000000000000000000000000000000000000000000000000000),
.INIT_04(256'h000000000000000000000000000000000000000
```

```
// The next set of INITP_xx are forr the parity bits
.INITP_00(256'h0000000000000000000000000000000000000000000000000000000000000000),
.INITP_01(256'h0000000000000000000000000000000000000000000000000000000000000000),
.INITP_02(256'h0000000000000000000000000000000000000000000000000000000000000000),
.INITP_03(256'h0000000000000000000000000000000000000000000000000000000000000000),
.INITP_04(256'h0000000000000000000000000000000000000000000000000000000000000000),
.INITP_05(256'h0000000000000000000000000000000000000000000000000000000000000000),
.INITP_06(256'h0000000000000000000000000000000000000000000000000000000000000000),
.INITP_07(256'h0000000000000000000000000000000000000000000000000000000000000000)
) BRAM_SINGLE_MACRO_inst0 (
// Note the data format for it to work.
// See RAMB18E1 primative below: get .DOADO({D[16:9],D[7:0]}), .DOPADOP({D[17],D[8]}) if just use "Data_out[17:0]"
.DO({Data_Out[17],Data_Out[15:8],Data_Out[16], Data_Out[7:0]}), // Output data, width defined by READ_WIDTH parameter
.ADDR(Add_In), // Input address, width defined by read/write port depth
.CLK(Clk_In), // 1-bit input clock
.DI(18'h3FFFF), // Input data port, width defined by WRITE_WIDTH parameter (not used)
.EN(1'b1), // 1-bit input RAM enable (permanently enabled)
.REGCE(1'b0), // 1-bit input output register enable (not used)
.RST(1'b0), // 1-bit input reset (not used)
.WE(2'b0) // Input write enable DISABLED, width defined by write port depth
);
```

uP16_Assembler

	G	H	J	K	L	M	O	Q	R	S	T	U
1			***OP CODE***	***R[rd]***	***R[rs]***	***D/I/TRG***	***Assembly Code***	***Code***	***Parity***	***Data***		***RAM Data Init***
2	00	>>	nop				nop	00000		0000		
3	01	>>	lli	r4		71	lli r4, #71	0E071	0	E071		
4	02	>>	nop				nop	00000		0000		
5	03	>>	lli	r1		1	lli r1, #1	0C801	0	C801		
6	04	>>	lli	r2		2	lli r2, #2	0D002		D002		
7	05	>>	lui	r4	r4	f9	lui r4, r4, #f9	124F9	4	24F9		
8	06	>>	add	r2	r1		add r2, r1	01101		1101		
9	07	>>	mov	r5	r3		mov r5, r3	02B0A	0	2B0A		
10	08	>>	addi	r3	r1	1	addi r3, r1, #1	15901		5901		
11	09	>>	addi	r6	r4	5	addi r6, r4, #5	17405	5	7405		
12	0A	>>	beq	r0	r0	4	beq r0, r0, #4	20004		0004		
13	0B	>>	nop				nop	00000	2	0000		
14	0C	>>	add	r4	r1		add r4, r1	02101		2101		
15	0D	>>	or	r2	r4		or r2, r4	01404	0	1404		
16	0E	>>	nop				nop	00000		0000		
17	0F	>>	sw	r4	r1	0	sw r4, r1, #0	0A100	0	A100	00	.INIT_00(256'hA100_0000_1404_2101_0000_0004_7405_5901_2B0A_1101_24F9_D002_C801_000
18	10	>>	sw	r5	r1	1	sw r5, r1, #1	0A901		A901		
19	11	>>	nop				nop	00000	0	0000		
20	12	>>	lw	r6	r1	0	lw r6, r1, #0	07100		7100		

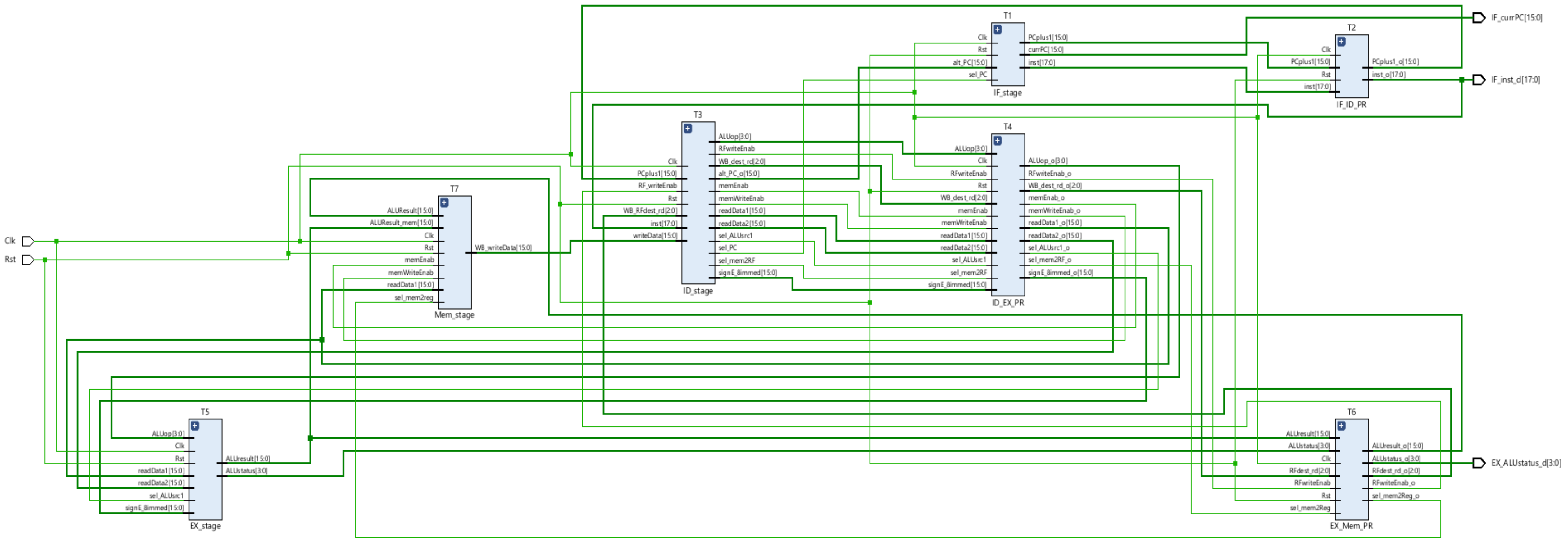
```
.DO_REG(0), // Optional output register (0 or 1)
//.INIT(36'h000000000), // Initial values on output port
.INIT_FILE ("NONE"),
.WRITE_WIDTH(18), // Valid values are 1-72 (37-72 only valid when BRAM_SIZE="36Kb")
.READ_WIDTH(18), // Valid values are 1-72 (37-72 only valid when BRAM_SIZE="36Kb")
//.SRVAL(36'h000000000), // Set/Reset value forr port output
.WRITE_MODE("WRITE_FIRST"), // "WRITE_FIRST", "READ_FIRST", or "NO_CHANGE"
.INIT_00(256'hA100_0000_1404_2101_0000_0004_7405_5901_2B0A_1101_24F9_D002_C801_0000_E071_0000),
.INIT_01(256'h0000_0000_1105_0A01_0000_5806_3202_D800_0000_2B01_1901_7901_0E01_7100_0000_A901),
.INIT_02(256'h0000_0000_0000_0000_0000_0000_0000_0000_0000_0000_0000_0000_0000_00DE_0C08_0000),
.INIT_03(256'h0000000000000000000000000000000000000000000000000000000000000000),
.INIT_04(256'h0000000000000000000000000000000000000000000000000000000000000000),
.INIT_05(256'h0000000000000000000000000000000000000000000000000000000000000000),
.INIT_06(256'h0000000000000000000000000000000000000000000000000000000000000000),
```

```
// The next set of INITP_xx are forr the parity bits
.INITP_00(256'h0000000000000000000000000000000000000000000000020002000000250400),
.INITP_01(256'h0000000000000000000000000000000000000000000000000000000000000000),
.INITP_02(256'h0000000000000000000000000000000000000000000000000000000000000000),
.INITP_03(256'h0000000000000000000000000000000000000000000000000000000000000000),
```

From uP16_Assembler.xlsx, take the codes for instruction and parity and paste inside the .INIT and .INITP

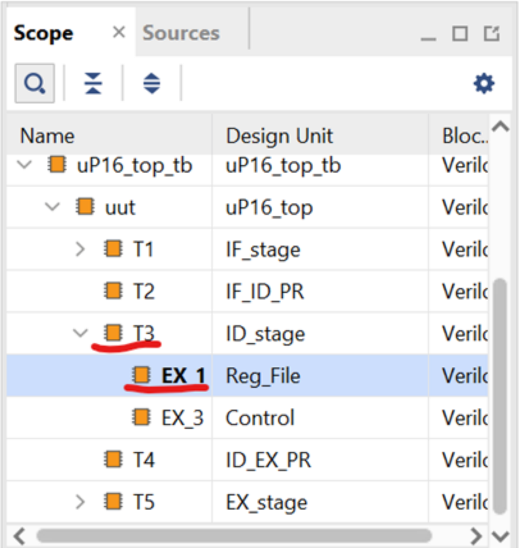
Note that 2nd instruction is still NOP and not changed to lli r3,ff

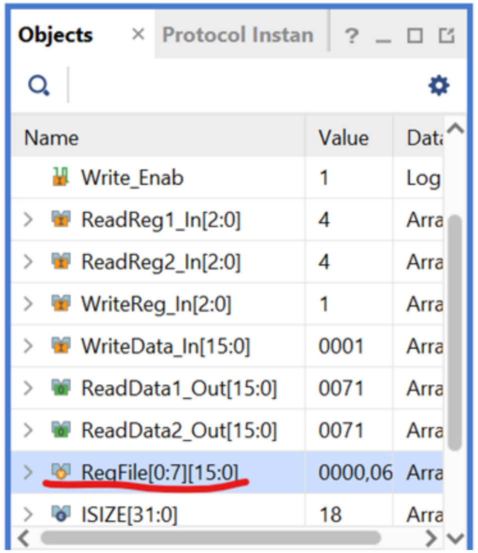
16 bit microprocessor- no data forwarding- RTL schematic

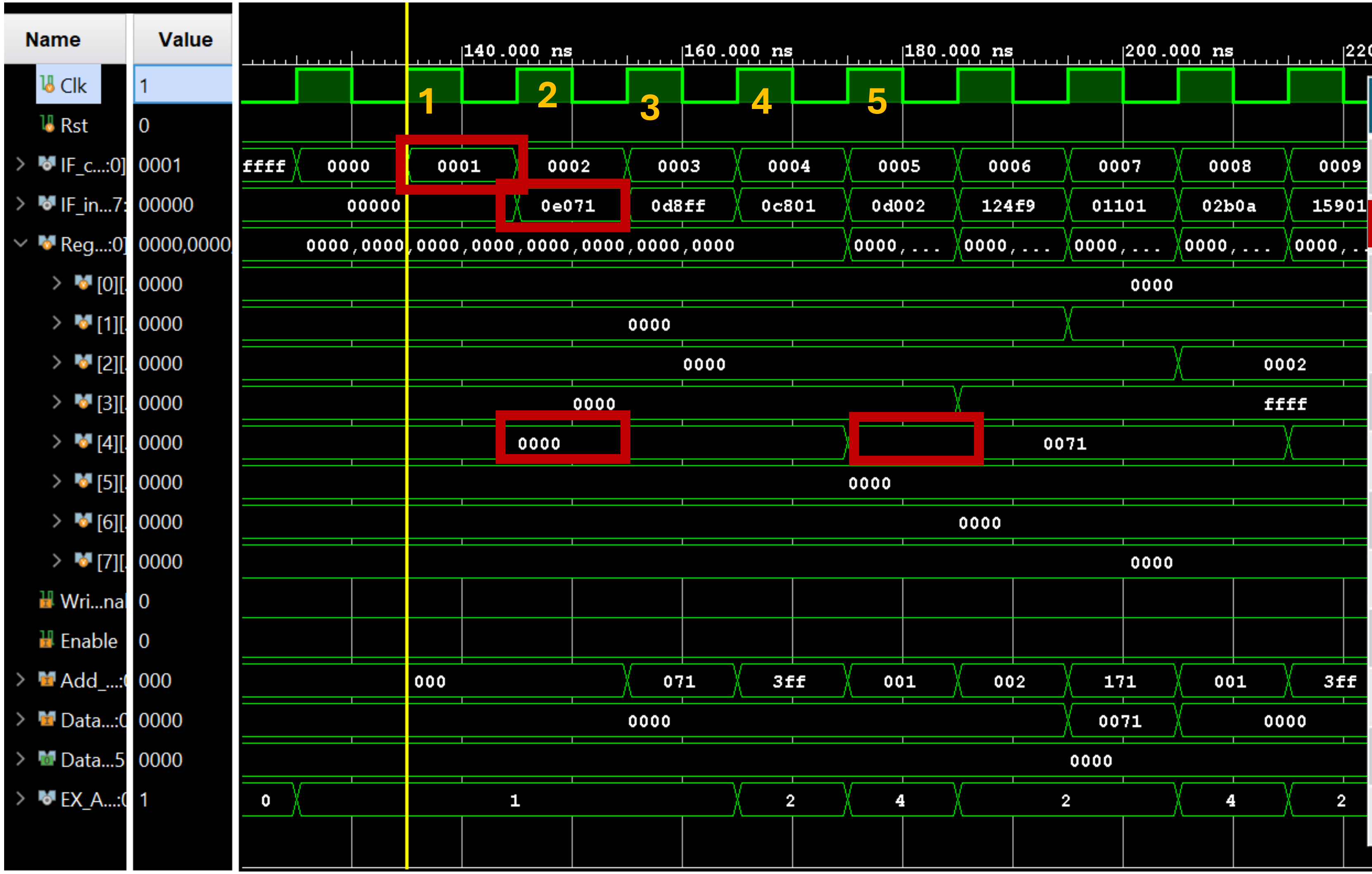


Task 1- Simulation using test bench (no propagation delay)

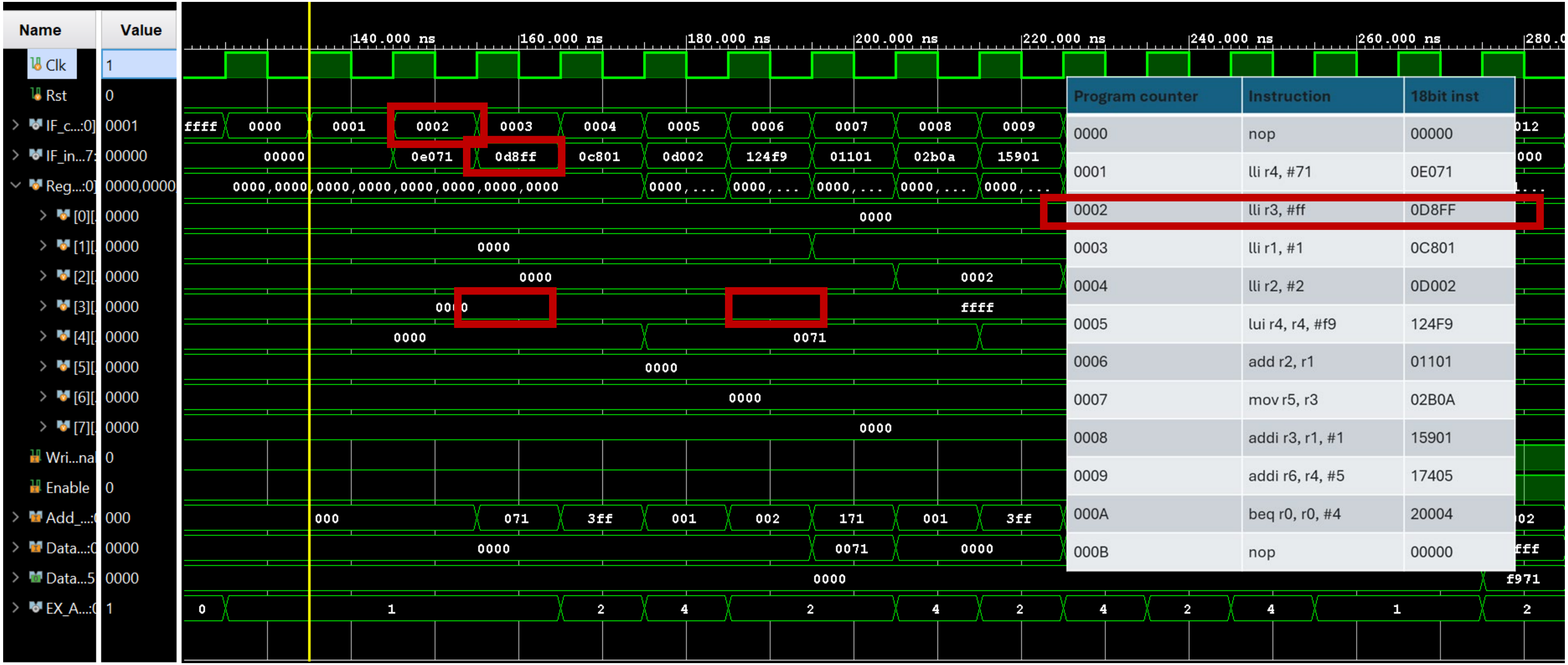
- After Synthesis, we need to do add the test bench as a simulation source and run the simulation
- Add the register file (RF), *RegFile [0:7,15:0]*, and Data memory, Write_Enab, Enable, Add_In, Data_in, Data_out simulation objects to the simulation.
- To do this open the *uP16_top_tb* instance in the "Scope" pane. Open *uut*, and select the sub-instance (the various instances match the module instantiation instances in uP_top.v) until you have the *RegFile* element. Then drag the simulation object (from the 'Objects" pane) to the simulation "Name" pane. Repeat for rest.



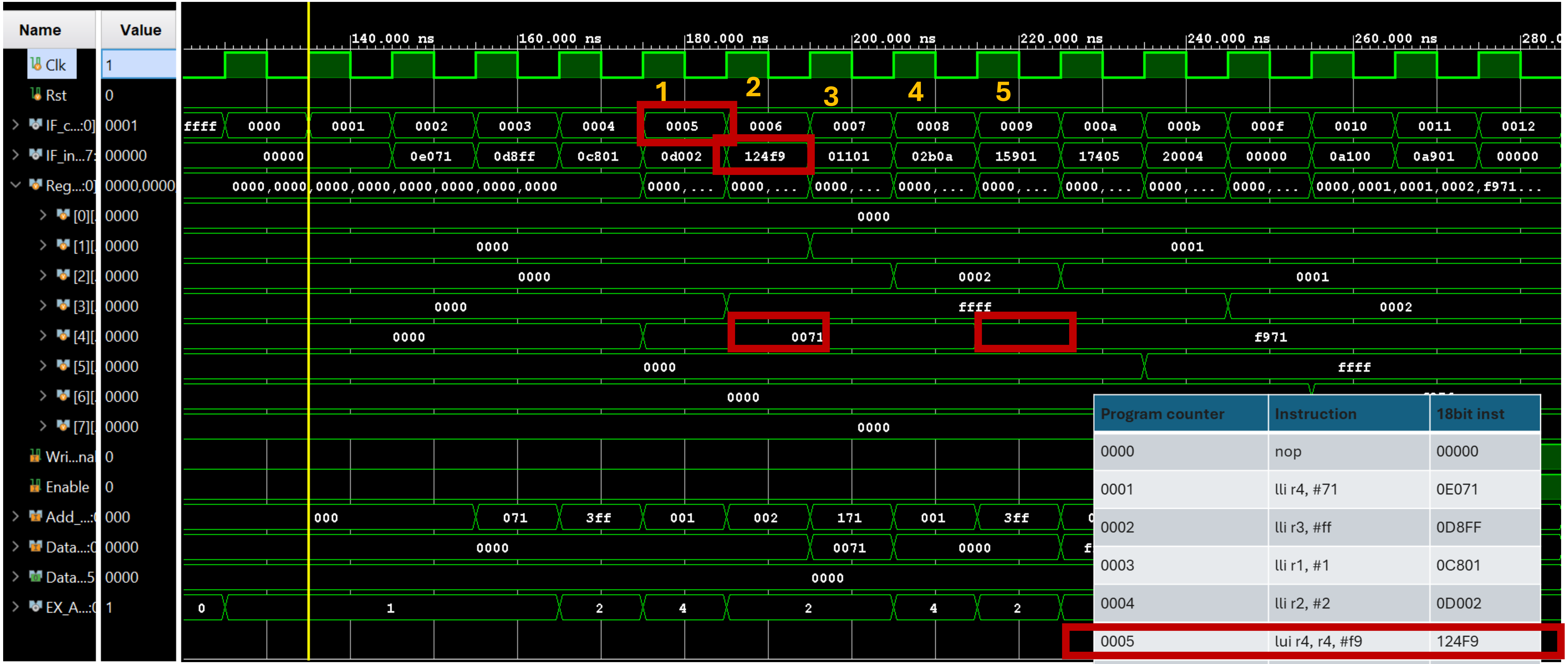




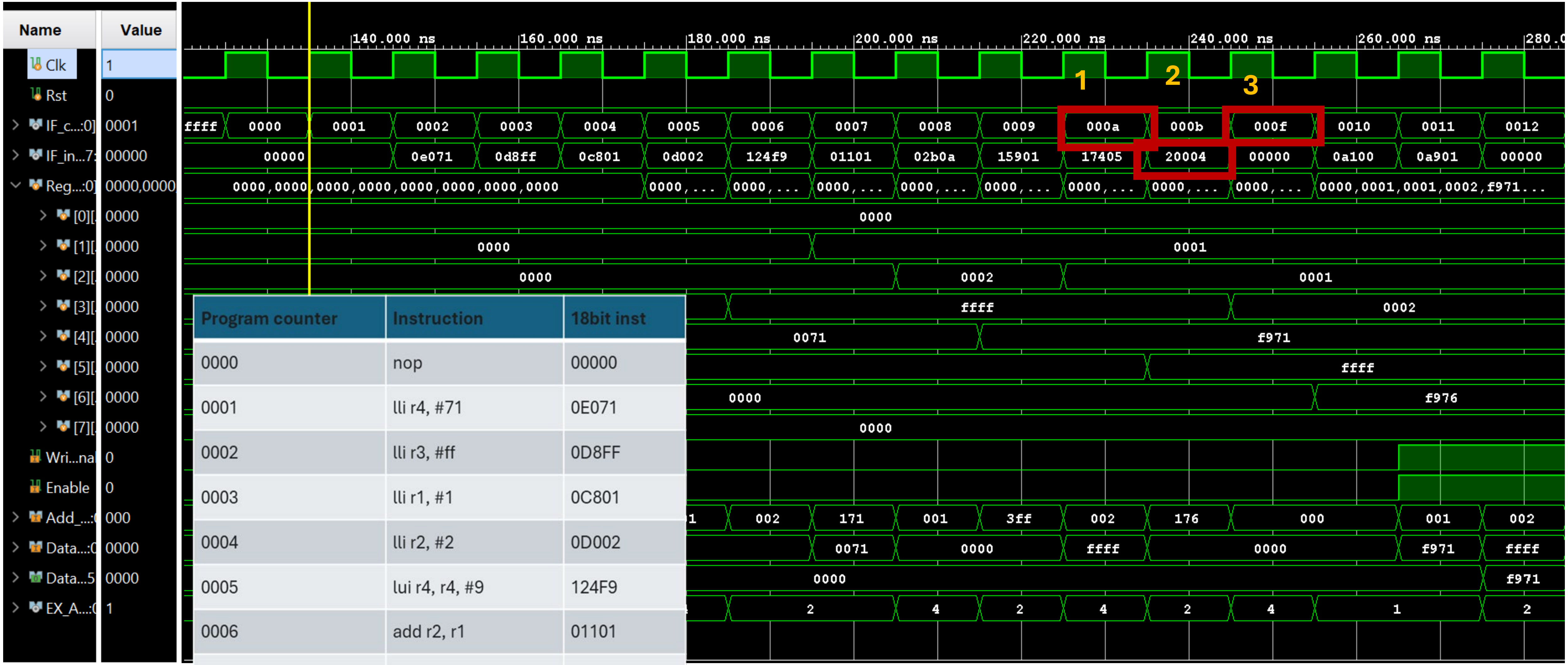
Program counter	Instruction	18bit inst
0000	nop	00000
0001	lli r4, #71	0E071
0002	lli r3, #ff	0D8FF
0003	lli r1, #1	0C801
0004	lli r2, #2	0D002
0005	lui r4, r4, #f9	124F9
0006	add r2, r1	01101
0007	mov r5, r3	02B0A
0008	addi r3, r1, #1	15901
0009	addi r6, r4, #5	17405
000A	beq r0, r0, #4	20004
000B	nop	00000



Name
Value
Clk
Rst
IF_c...:0]
IF_in...7:
Reg...:0]
Wri...nal
Enable
Add_...:
Data...:
Data...5
EX_A...:
140.000 ns
160.000 ns
180.000 ns
200.000 ns
220.000 ns
240.000 ns
260.000 ns
Program counter | Instruction | 18bit inst
0000 | nop | 00000
0001 | lli r4, #71 | 0E071
0002 | lli r3, #ff | 0D8FF
0003 | lli r1, #1 | 0C801
0004 | lli r2, #2 | 0D002
0005 | lui r4, r4, #f9 | 124F9
0006 | add r2, r1 | 01101
0007 | mov r5, r3 | 02B0A
0008 | addi r3, r1, #1 | 15901
0009 | addi r6, r4, #5 | 17405
000A | beq r0, r0, #4 | 20004
000B | nop | 00000



Program counter	Instruction	18bit inst
0000	nop	00000
0001	lli r4, #71	0E071
0002	lli r3, #ff	0D8FF
0003	lli r1, #1	0C801
0004	lli r2, #2	0D002
0005	lui r4, r4, #f9	124F9
0006	add r2, r1	01101
0007	mov r5, r3	02B0A
0008	addi r3, r1, #1	15901
0009	addi r6, r4, #5	17405
000A	beq r0, r0, #4	20004
000B	nop	00000



Program counter	Instruction	18bit inst
0000	nop	00000
0001	lli r4, #71	0E071
0002	lli r3, #ff	0D8FF
0003	lli r1, #1	0C801
0004	lli r2, #2	0D002
0005	lui r4, r4, #9	124F9
0006	add r2, r1	01101
0007	mov r5, r3	02B0A
0008	addi r3, r1, #1	15901
0009	addi r6, r4, #5	17405
000A	beq r0, r0, #4	20004
000B	nop	00000

000a+1+4= 000f

(Task 1)Simulation window with upation of register file as explained earlier

(Task 2) Same simulation window with propagation delay shown

(Task3) Implementation uP16 design on the Basys3 board